Állapotgép

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

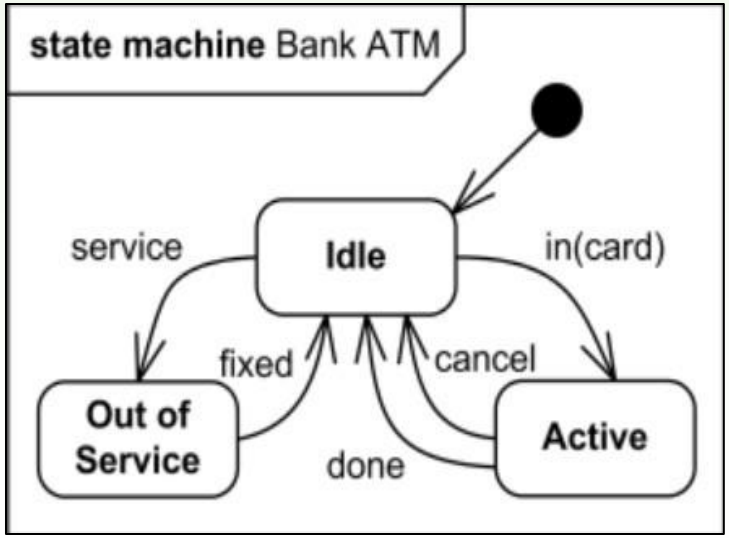
Objektum életciklusa

- Az objektum életciklusa során:
  - létrejön: példányosodik (konstruktor)
  - működik: más objektumokkal kommunikál, azaz szinkron vagy aszinkron módon üzeneteket küldenek egymásnak, vagy valamilyen más esemény következik be, és ennek során megváltoznak az adatai
  - megszűnik: megsemmisül (destruktor)
- Az objektumnak működése során különböző állapotai (*state*) lehetnek: az állapot (fizikai állapot) az objektum adatai által felvett értékeknek együttese.
- Gyakran egy állapoton nem a fizikai állapotot, hanem több, valamilyen szempontból közös tulajdonságú fizikai állapot összességét értjük (logikai állapot).

Állapot-átmenet gráf

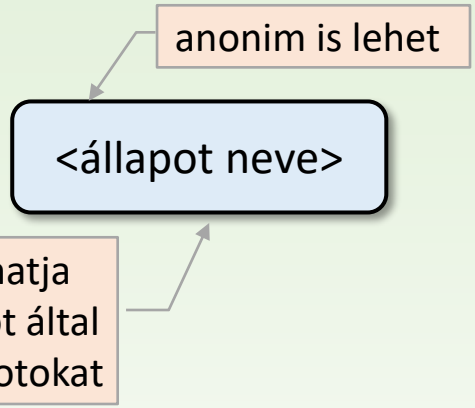
- Egy objektum életciklusát – logikai állapotról logikai állapotra változó működését – egy állapot-átmenet gráffal ábrázolhatjuk.
- Ez egy olyan irányított gráf, amelynek csúcsai az objektum logikai állapotait, irányított élei pedig az állapotok közötti átmeneteket jelölik. Az állapotokhoz is, és az átmenetekhez is tartozhatnak végrehajtandó tevékenységek.

- Egy objektumnak egyszerre csak egy állapota lehet aktív. Az aktív állapot egy esemény (pl. az objektumnak küldött metódushívás vagy szignál) hatására egyértelmű módon változhat meg. Az új aktív állapot a régiből kivezető élek által mutatott állapotok valamelyike.



Állapotok

- Az állapotokat lekerekített sarkú téglalap jelöli:

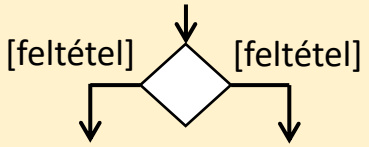


Pszeudo állapotok:

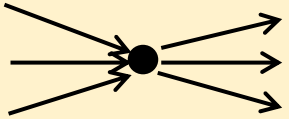
- kezdő állapot
- végállapot



- elágazás (choice)



- csomópont (junction)



Tevékenységek

- Az állapotgép digramban egy tevékenység végrehajtására sor kerülhet
  - egy állapot-átmenet során
  - egy állapothoz kapcsolódóan



Belépési tevékenység: akkor hajtódik végre (és mindig be is fejeződik), amikor egy átmenet révén az állapot aktív lesz.

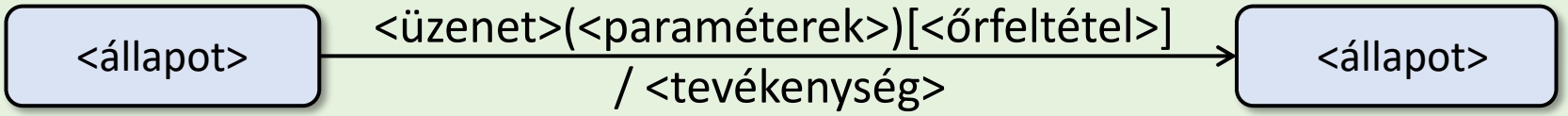
Belső tevékenység: addig tart, amíg az állapot aktív (ez a tevékenység lehet folyamatos vagy befejeződő)

Kilépési tevékenység: akkor hajtódik végre (és mindig be is fejeződik), amikor egy átmenet révén másik állapot válik aktívvá.

Belső átmenet tevékenysége: ez egy reflexív átmenet tevékenysége, amely esetén az állapotnak sem a be-, sem a kilépési tevékenysége nem hajtódik végre.

Egy átmenet megvalósulásakor megszakad a belső tevékenység, és egymás után végrehajtódik a kilépési tevékenység, az átmenet tevékenysége, végül az új állapot belépési tevékenysége.

Állapot-átmenet

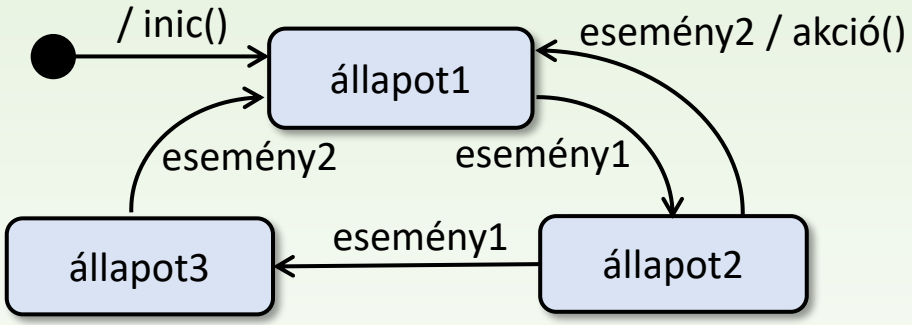


- Egy objektumból induló átmenet valamilyen esemény (*event*) hatására következik be. Ez az esemény lehet
  - üzenet (*trigger*), amelynek paraméterei is lehetnek
    - objektum metódusának hívása
    - objektumnak küldött szignál észlelése
  - objektum belső tevékenységének befejeződése
  - őrfeltétel (*guard*) teljesülése, amely vagy egy logikai állítás (*when*), amely az üzenet paramétereitől és az objektum adattagjaitól függ, vagy egy időhöz kötött várakozási feltétel (*after*)
- Az átmenethez rendelt tevékenység (az objektum adattagjaival és a kiváltó üzenet paramétereivel operáló program)
- Egy átmenet lehet reflexív (az aktív állapotba visszatérő) is.

egy átmenet speciálisan lehet összetett: állapothalmazból állapothalmazba

Állapot-átmenet tábla

- Az állapotgép diagram működése leírható egy táblázattal is.



állapot / **esemény**	állapot1 / start: inic()	állapot2	állapot3
esemény1	állapot2	állapot3	
esemény2		állapot1 / akció()	állapot1

Állapotgép megvalósítása

állapot / esemény	állapot1 / start: inic()	állapot2	állapot3
esemény1	állapot2	állapot3	
esemény2		állapot1 / akció()	állapot1

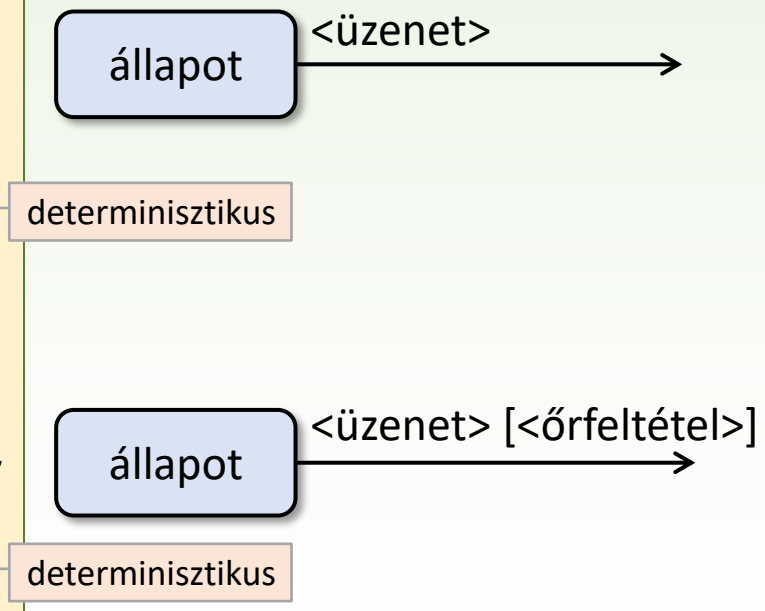
```
inic()
állapot := állapot1
while állapot≠stop loop
  switch esemény
    case esemény1:
      switch állapot
        case állapot1 : állapot := állapot2
        case állapot2 : állapot := állapot3
        case állapot3 :
    endswitch
    case esemény2:
      switch állapot
        case állapot1 :
        case állapot2 : akció()
                        állapot := állapot1
        case állapot3 : állapot := állapot1
      endswitch
  endswitch
endloop
```

```
inic()
állapot := állapot1
while állapot≠stop loop
  switch állapot
    case állapot1:
      switch esemény
        case esemény1 : állapot := állapot2
      endswitch
    case állapot2:
      switch esemény
        case esemény1 : állapot := állapot3
        case esemény2 : akció()
                        állapot := állapot1
      endswitch
    case állapot3:
      switch esemény
        case esemény2 : állapot := állapot1
      endswitch
  endswitch
endloop
```

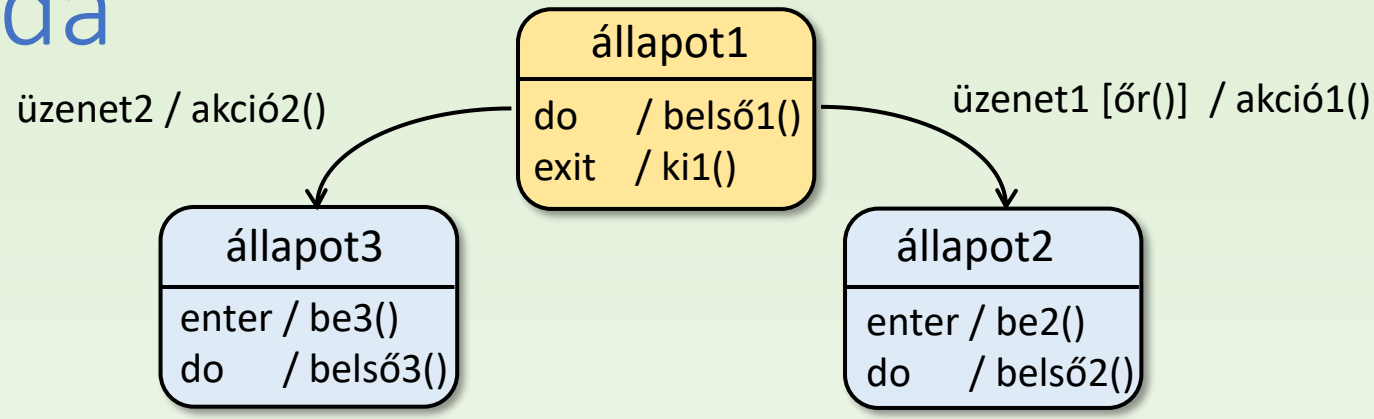
Állapot-átmenetek szemantikája I.

Amikor az átmenet egy üzenet váltja ki (metódus hívás vagy szignál küldés)

- Őrfeltétel hiányában az átmenet azonnal megvalósul.
  Ugyanazon állapotból nem vezethet ki azonos üzenettel több őrfeltétel nélküli él.
- Őrfeltétel esetén az átmenet akkor valósul meg, ha őrfeltétel éppen teljesül az üzenet beérkezésekor az (az átmenet nem várakozik az eseményre.)
  Ugyanazon üzenet csak akkor tartozhat egy állapotból kivezető több élhez is, ha azok őrfeltételei diszjunktak.



Példa




```
while állapot≠stop loop
  switch állapot
    case állapot1:
      switch esemény
        case üzenet1 : if őr() then
                          stop(belső1()); ki1();
                          akció1()
                          állapot := állapot2
                          be2(); start(belső2())
                       endif
         case üzenet2 :  stop(belső1()); ki1();
                         akció2()
                         állapot := állapot3
                         be3(); start(belső3())
      endswitch
    ...
  endswitch
endloop
```

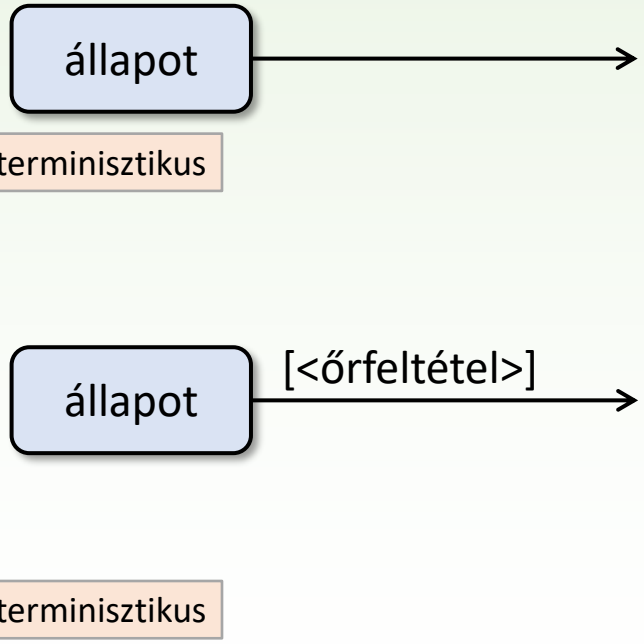
megszakad a belső tevékenység

elindul a belső tevékenység

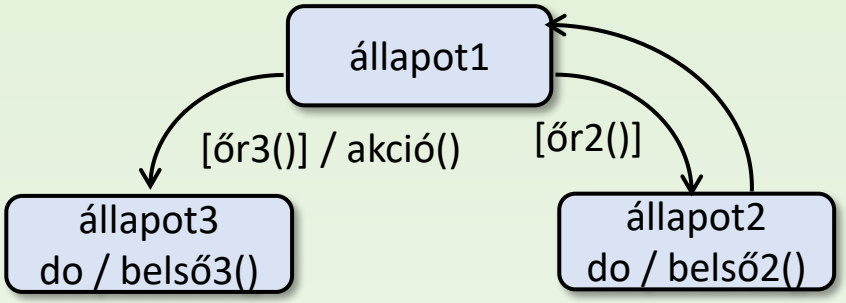
Állapot-átmenetek szemantikája II.

Amikor az átmenethez nem tartozik üzenet, akkor az aktív állapot belső tevékenységének befejeződése váltja ki az átmenetet.

- Őrfeltétel hiányában az átmenet azonnal megvalósul.
  Ilyen átmenetből állapotonként csak egy lehet.
- Őrfeltétel esetén az átmenet várakozik arra, hogy az őrfeltétel igaz legyen, de várakozás közben sor kerülhet egy másik állapot-átmenetre.
  Az egy állapotból kivezető nem üzenetre megvalósuló átmeneteknek diszjunkt őrfeltételekkel kell rendelkezniük.



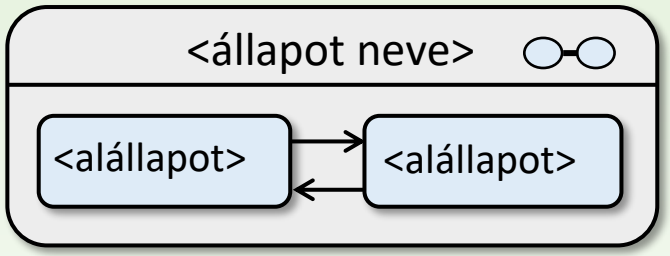
Példa



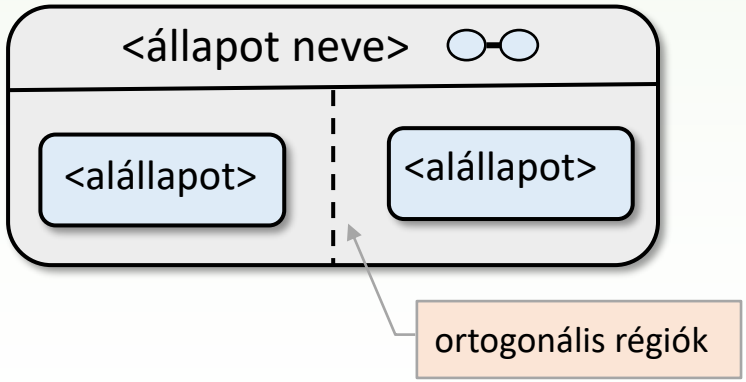

```
while állapot ≠ stop loop
   switch állapot
      case állapot1 : wait ( őr2() or őr3() )
                      if (őr2() then
                         állapot := állapot2
                      elseif (őr3() then
                          akció()
                          állapot := állapot3
                      endif
       case állapot2 : belső2()
                       állapot := állapot1
       case állapot3 : belső3()
                  ...
   endswitch
endloop
```

Hierarchikus állapotok

Szekvenciális:



Párhuzamos:

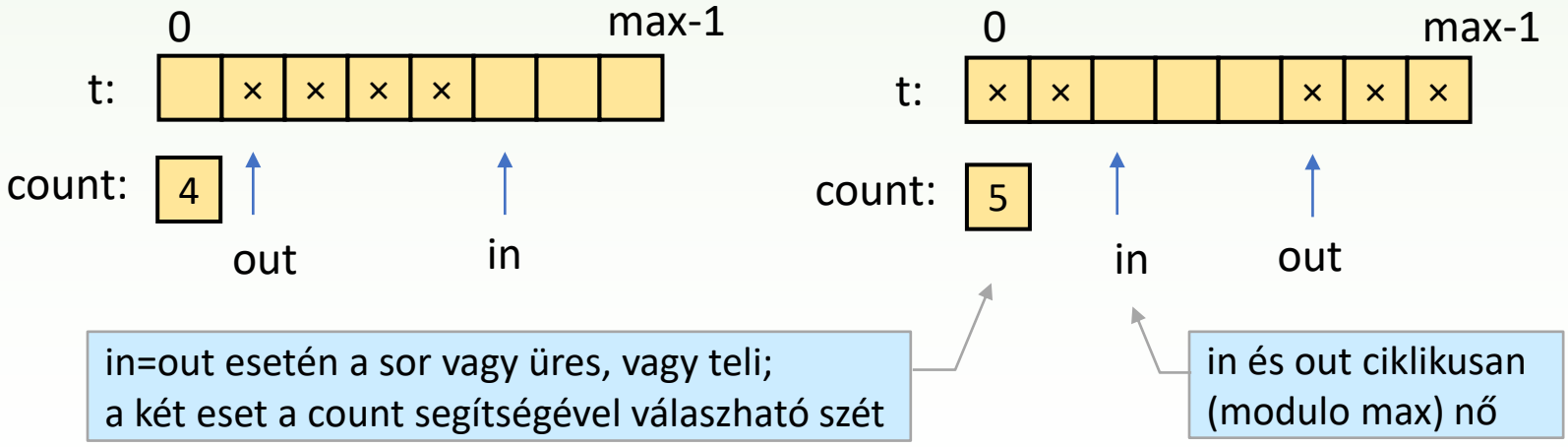


Pszeudo állapotok:

- belépés (entry)
- kilépés (exit)
- megszüntetés
- shallow history
- deep history
- szétágazás (fork)
- összefutás (join)

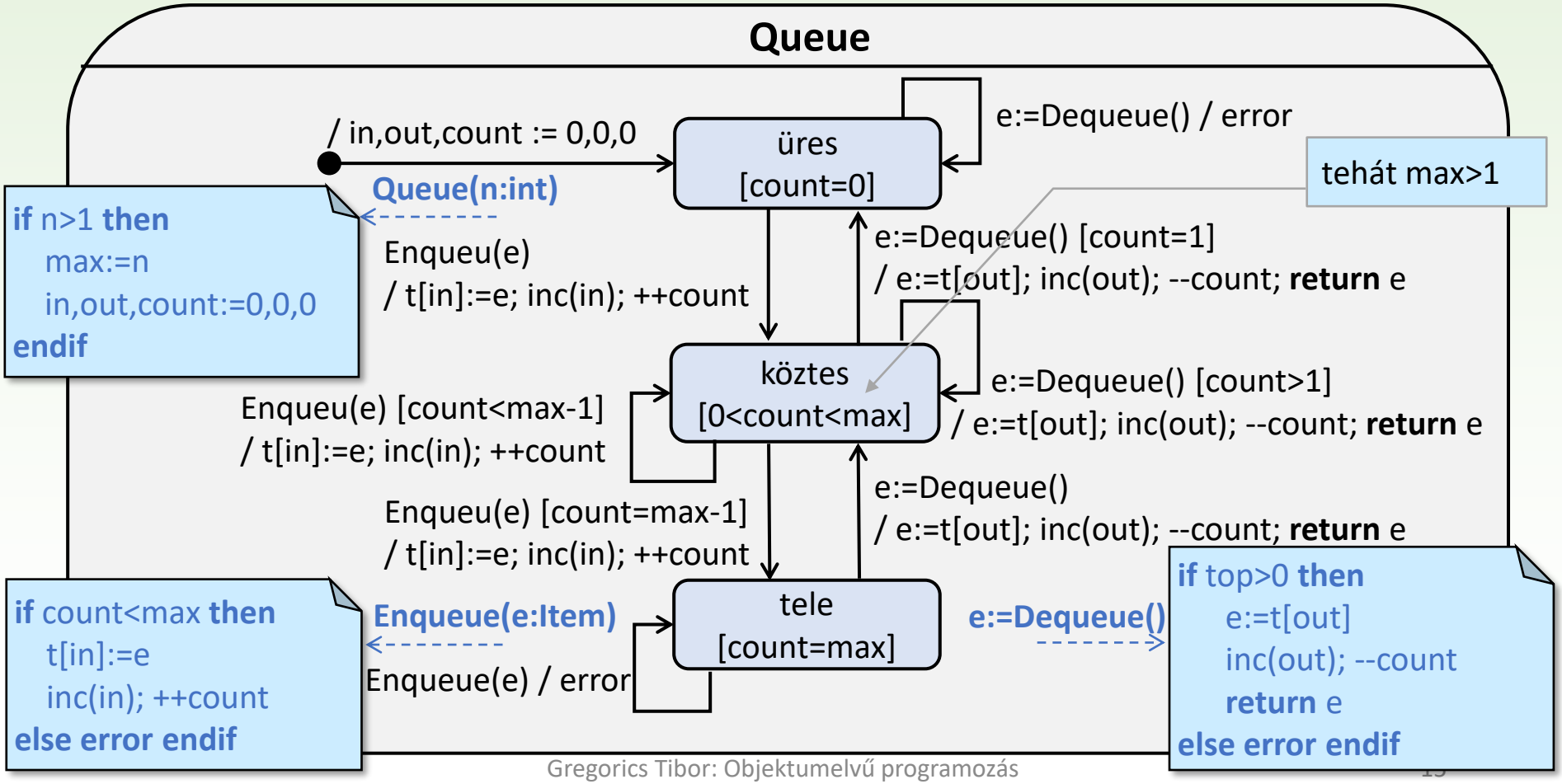
Példa: Korlátos sor

- Készítsünk egy tárolót, amely sorként (FIFO) működik az Enqueue() és Dequeue() műveletekkel.
- Az előre nem meghatározott típusú elemeket egy rögzített méretű (max:int) tömb (t : array[0..max-1] of Item) tárolja, amelynek az eleje és vége indexei (in, out:int) közötti része felel meg a sorban tárolt elemeknek. Ezek darabszámát (count:int) külön is eltároljuk.



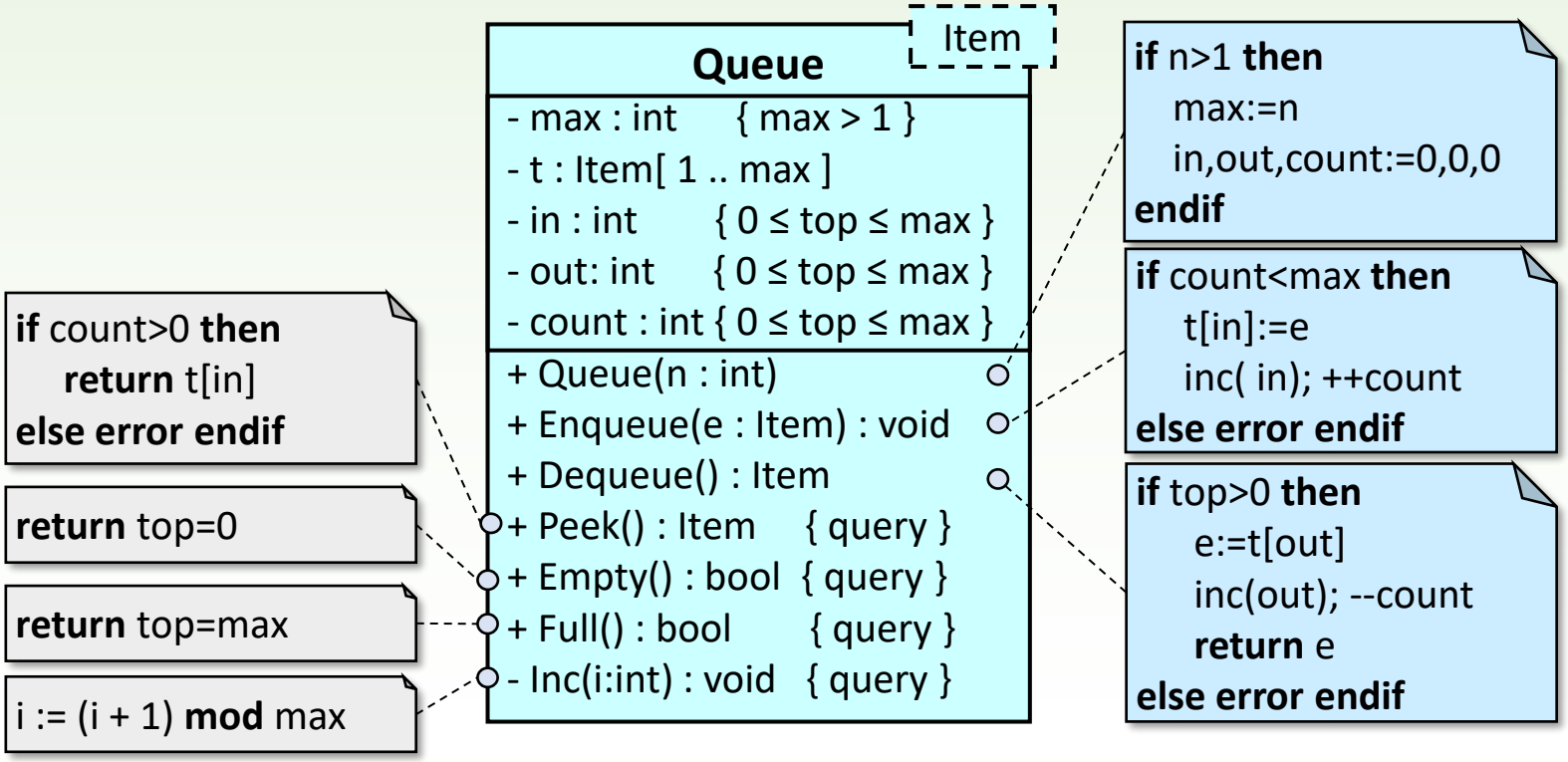
Korlátos sor állapotgépe

- Az állapotgép modellje segíti a metódusok tervezését.
- Három állapotot vezetünk be, amelyek között a sor Enqueue() és Dequeue() műveletei hatására következhet be majd átmenet.



Korlátos sor osztály diagramja

- Az állapotgép elemzésének eredményeként kapjuk meg az osztály metódusait.
- A megvalósítás során az állapotgépet nem kell önállóan megvalósítani: ennek működése a metódusok call-back függvényként való hívásain keresztül érvényesül.



Állapotgép megvalósítás

Garázskapu

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

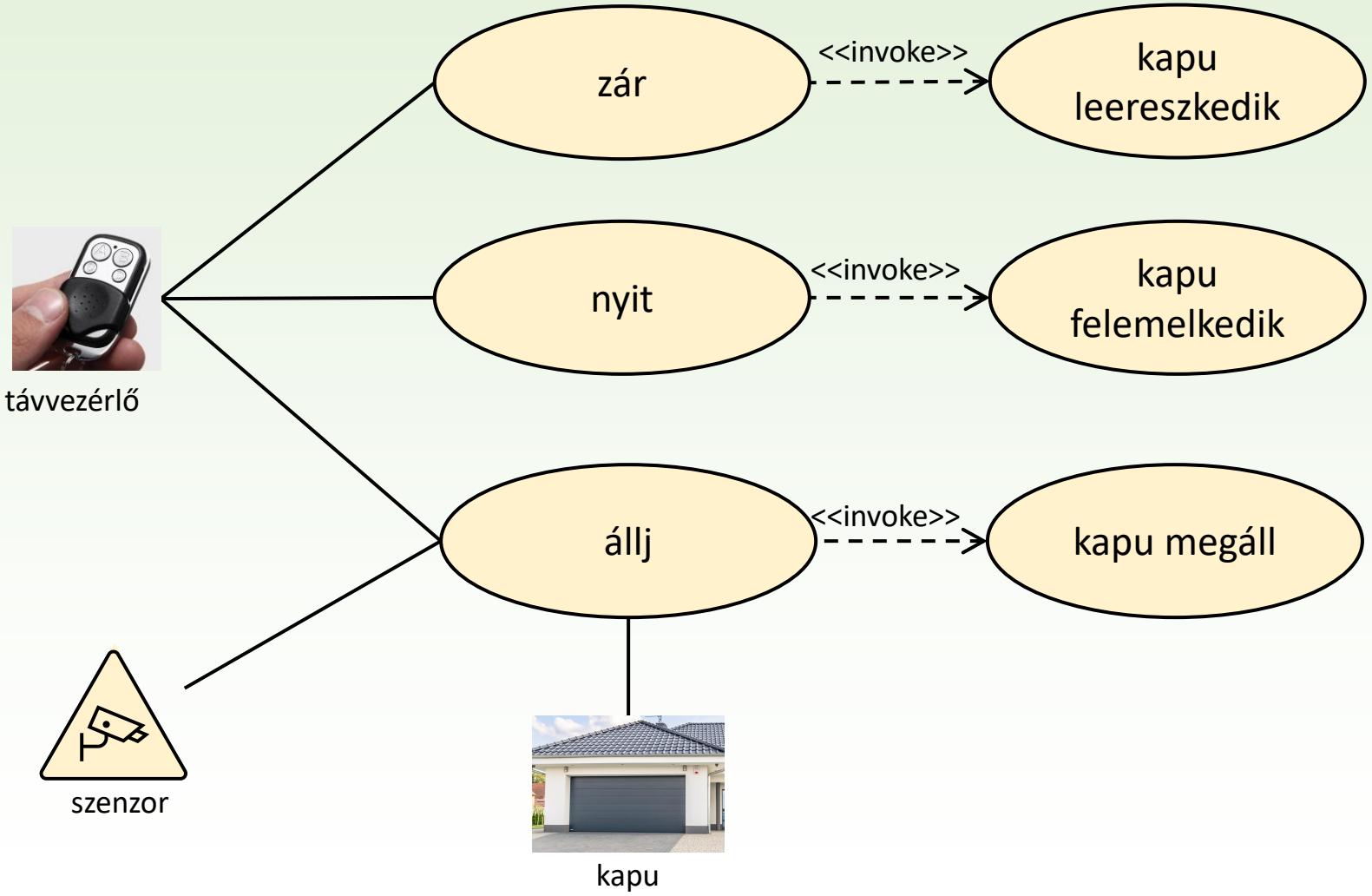
Feladat

Tervezzük meg, és implementáljuk egy függőlegesen nyíló garázskapu működését!

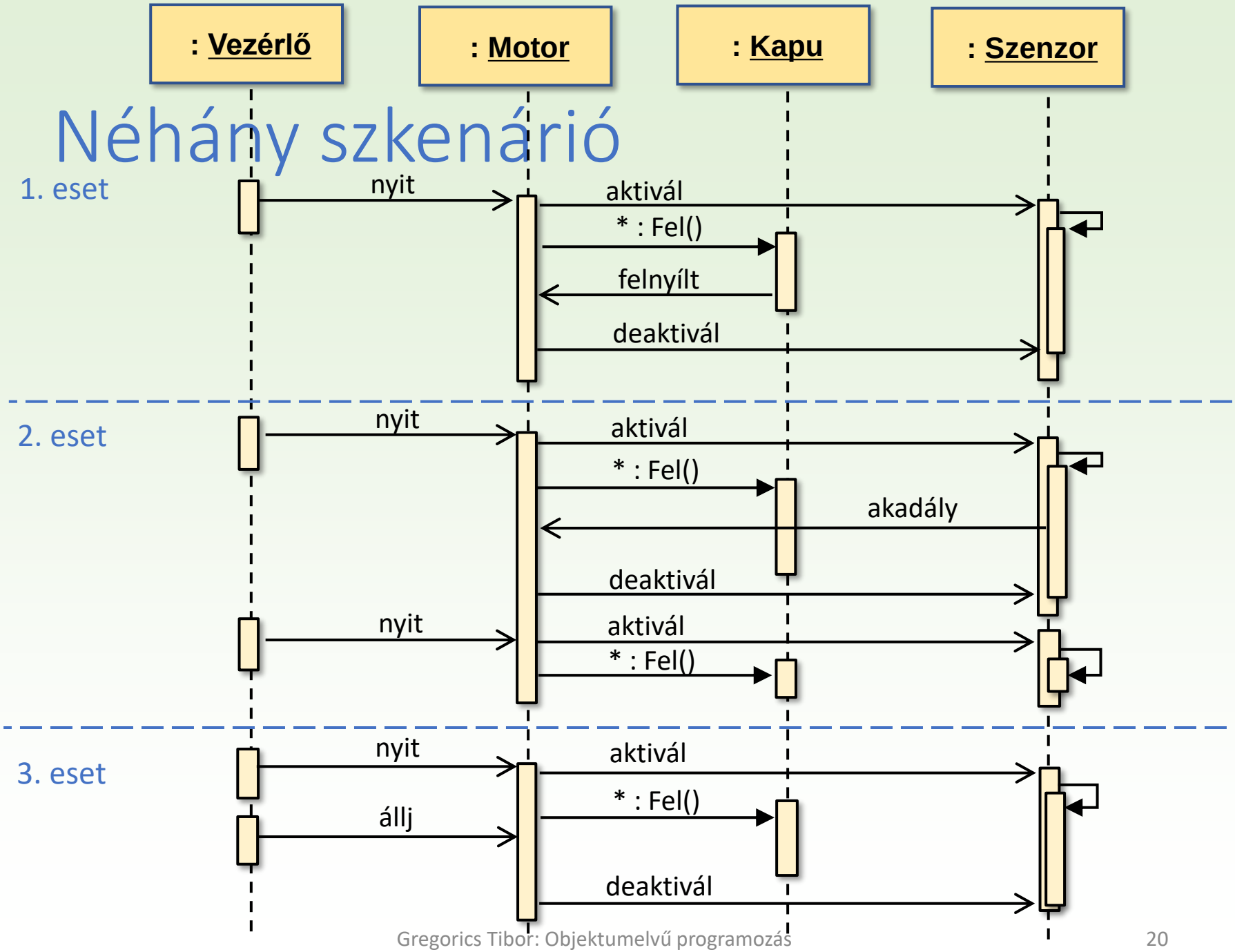
- A garázskaput mozgató motort egy távirányító vezérli. A távirányító három féle parancsot adhat a kapunak: záródj, nyíljál, állj meg.
- A garázskapu a legutoljára kiadott parancs szerint működik, de magától megáll, ha teljesen kinyílt, vagy lecsukódott, illetve ha akadályba ütközik:
  - a kapu jelzi a motornak, ha teljesen felnyílt vagy lecsukódott
  - az akadályt egy érzékelő jelzi a motornak



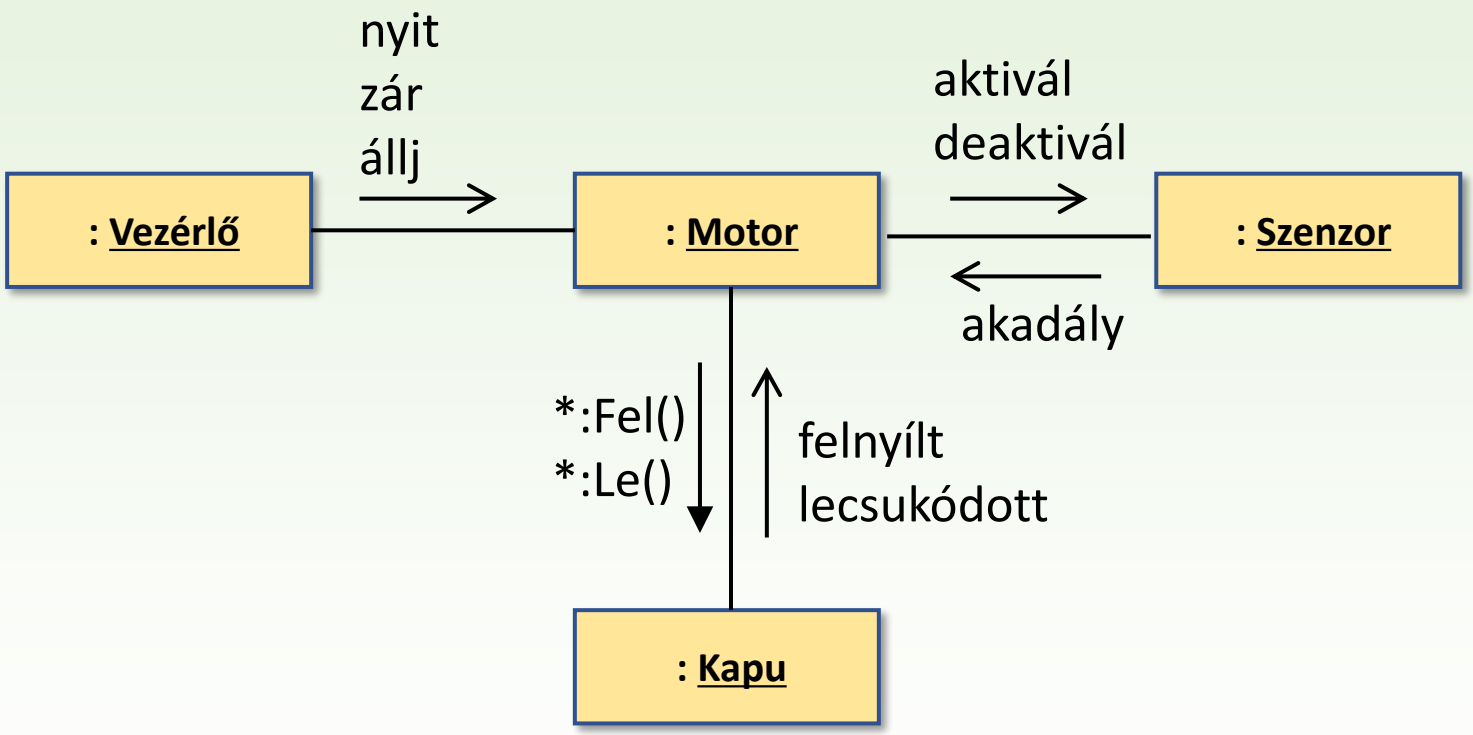
Használati eset diagram



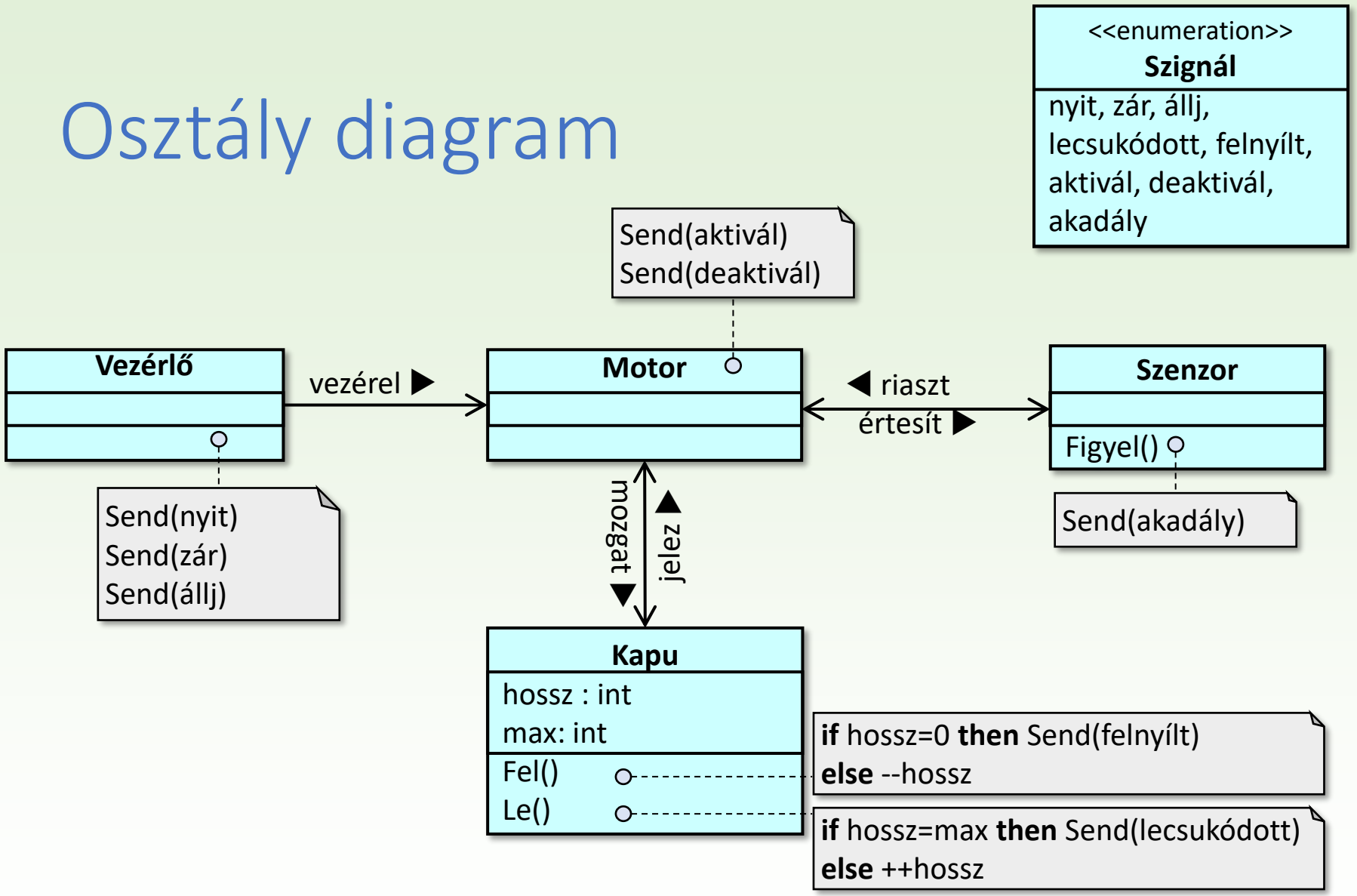
Néhány szkenárió



Kommunikációs diagram



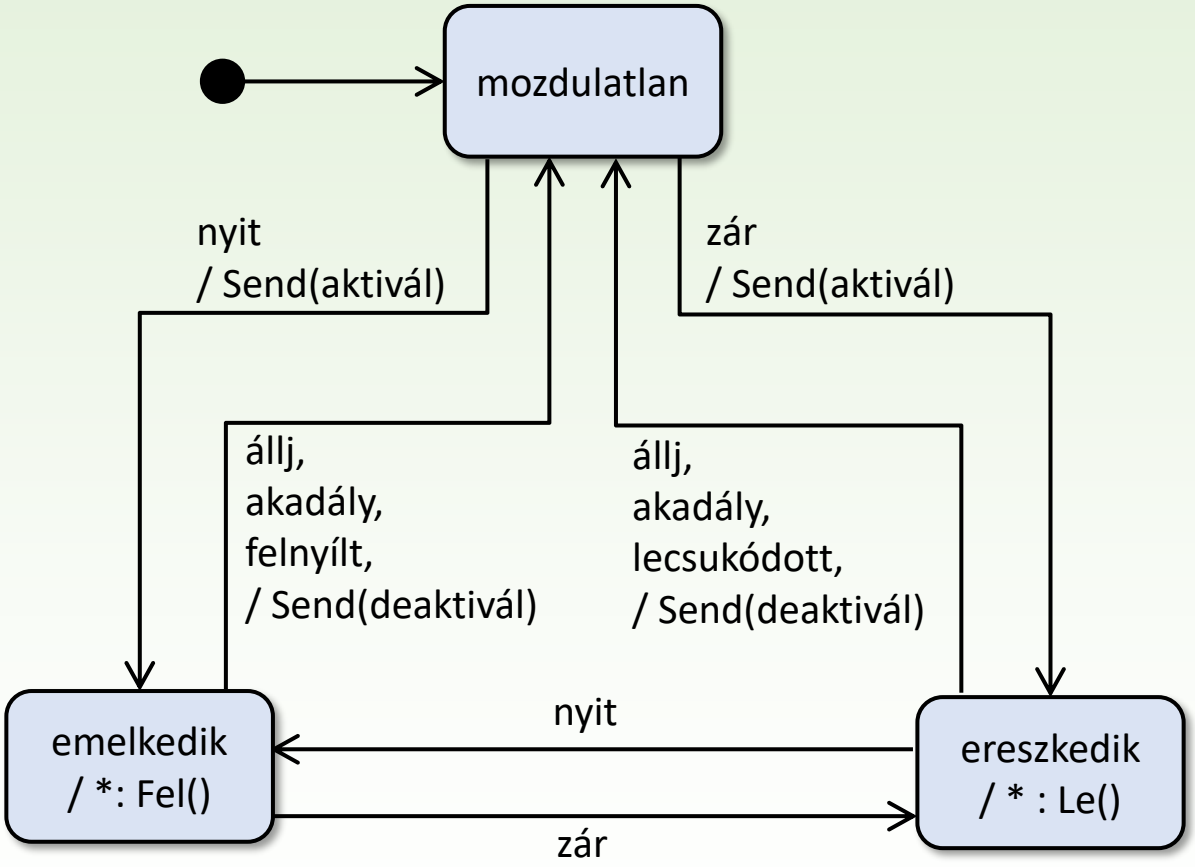
Osztály diagram



Állapotgépek

- Az objektumok egy része (motor, szenzor) aszinkron dolgozza fel a nekik küldött szignálokat, mások (kapu) szinkron metódus-hívásokat kapnak.
  - a motor, miközben irányítja a kaput, a vezérlőtől és a szenzortól kaphat aszinkron szignálokat, amelyeket a beérkezésük sorrendjében kell feldolgoznia: ezt állapotgéppel jellemezhetjük
  - a szenzor két állapota közül az aktív állapotban figyeli az akadályokat, amelyet a motor aszinkron módon állíthat le.
  - a kaput szinkron hívásokkal vezérli a motor; ezért külön állapotgépet nem kell készítünk hozzá.
  - a vezérlő egység csak a felhasználó jelzéseit továbbítja a motor és a szenzor felé; nincsenek megkülönböztethető állapotai.

Motor állapotgép diagramja



<<enumeration>>
Állapot1
emelkedik,
ereszkedik,
mozdulatlan

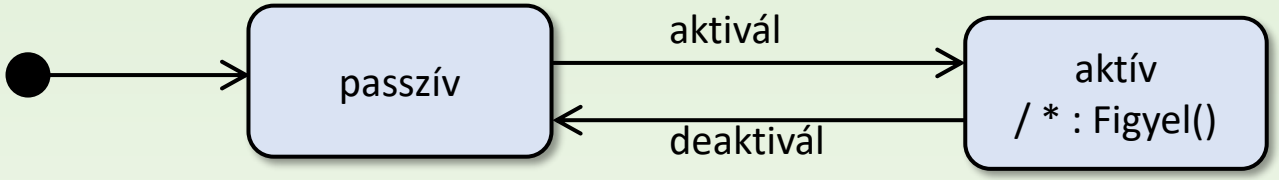
<<enumeration>>
Szignál
nyit, zár, állj,
lecsukódott, felnyílt,
aktivál, deaktivál,
akadály

Ez a megoldás itt azért jó, mert az állapot belső tevékenysége ugyanazon lépés ciklikus ismétlése addig, amíg egy üzenet ezt meg nem szakítja.

állapot / esemény	emelkedik	ereszkedik	mozdulatlan
nyit		emelkedik	emelkedik /Send(aktivál)
zár	ereszkedik		ereszkedik /Send(aktivál)
állj, akadály lecsukódott, felnyílt	mozdulatlan /Send(deaktivál)	mozdulatlan /Send(deaktivál)	
nincs	/ kapu.Fel()	/ kapu.Le()	

```
switch állapot
   case emelkedik:
       switch esemény
            case nincs, nyit : kapu.Fel()
            case zár:  állapot := ereszkedik
            case állj, lecsukódott, felnyílt, akadály: állapot := mozdulatlan; Send(deaktivál);
       endswitch
   case ereszkedik:
       switch esemény
            case nyit:  állapot := emelkedik
            case nincs, zár : kapu.Le()
            case állj, lecsukódott, felnyílt, akadály: állapot := mozdulatlan; Send(deaktivál);
       endswitch
   case mozdulatlan:
      switch esemény
            case nyit:  állapot := emelkedik; Send(aktivál);
            case zár:   állapot := ereszkedik; Send(aktivál);
       endswitch
endswitch
```

Szenzor állapotgépe



állapot / esemény	aktív	passzív
aktivál		aktív
deaktivál	passzív	
nincs	Figyel()	

<<enumeration>>
Állapot2
aktív,
passzív

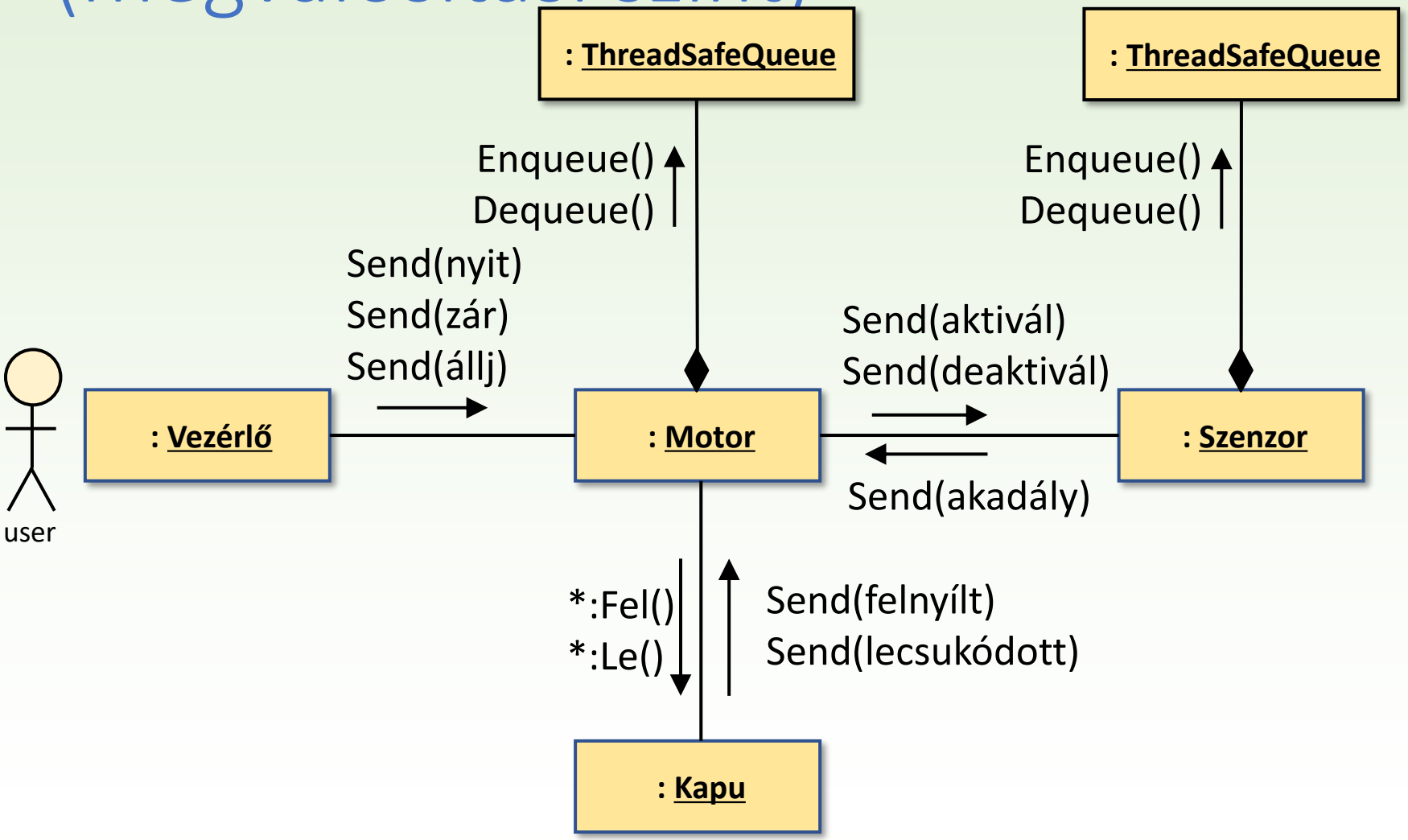
<<enumeration>>
Szignál
nyit, zár, állj, lecsukódott, felnyílt, aktivál, deaktivál, akadály

```
switch (akt) {
      case passzív:
            switch (szignál) {
                  case aktivál: akt := aktív; Figyel()
            }
      case aktív:
             switch (szignál) {
                  case deaktivál: akt := passzív;
                  case nincs: Figyel()
            }
}
```

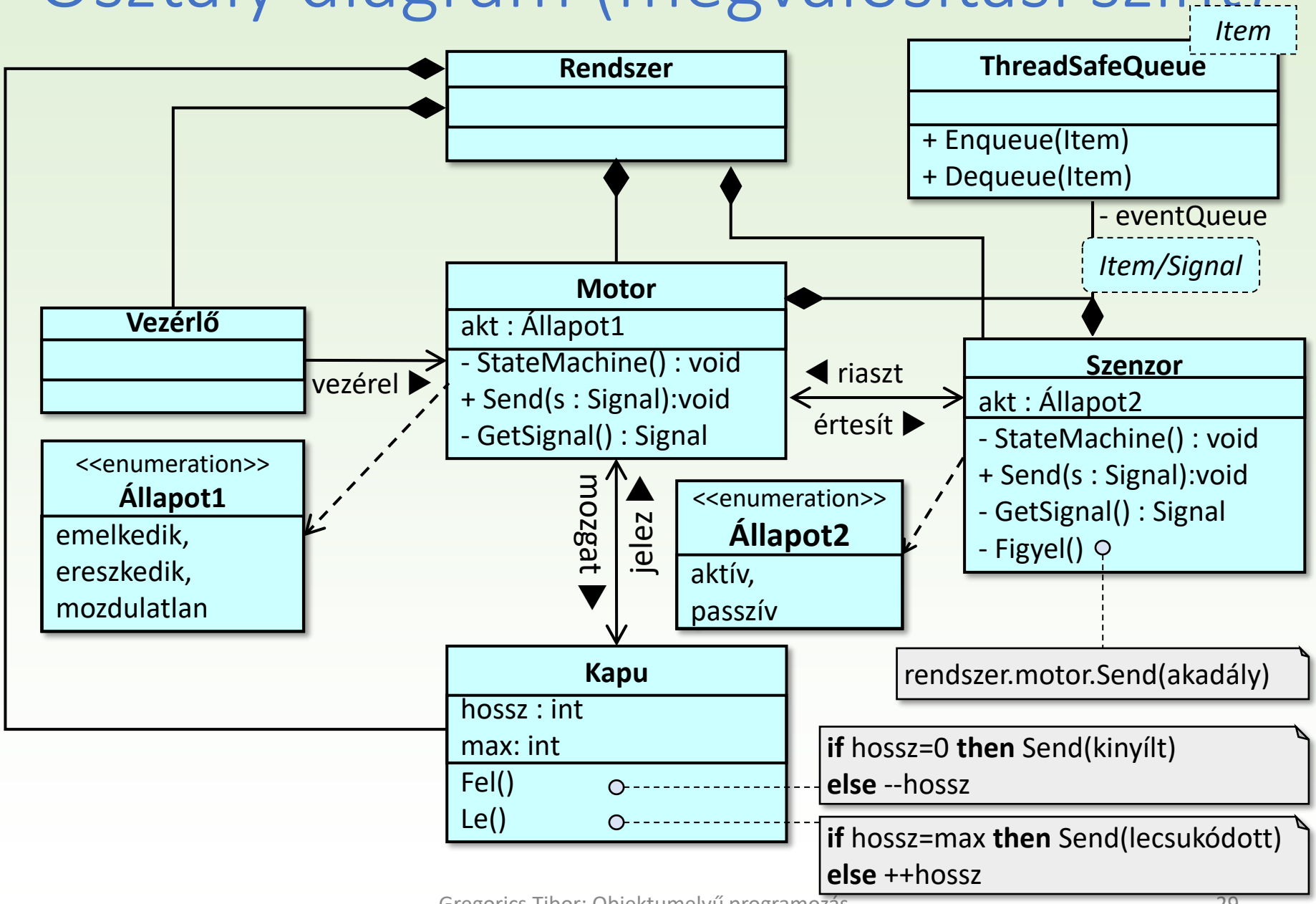
Aszinkron üzenetekkel vezérelt állapotgép megvalósítása

- A megvalósítás során az üzeneteket aszinkron módon feldolgozó állapotgépeket az üzenetet küldő szálaktól eltérő szálon kell futtatni.
- Az objektum a több irányból érkező szignálokat egy eseménysorban gyűjti.
  - A küldő objektumok a fogadó objektum Send() metódusának hívásával tehetnek bele egy szignált a fogadó eseménysorába; ez a metódus tehát a küldő objektum szálán hajtódik végre.
  - A fogadó objektum saját szálán futó állapotgépe a getSignal() metódussal vesz ki egy szignált az eseménysorból.
  - Az eseménysor betesz és kivesz műveleteit szinkronizálni kell: kölcsönösen kizárásos módon használhatják a sort. Ráadásul üres sor esetén a kivesz műveletet várakozó utasítással blokkolni kell.

Kommunikációs diagram (megvalósítási szint)



Osztály diagram (megvalósítási szint)



Rendszer osztálya

```
class System
{
public:
    System();
    ~System();

    Engine *_engine;
    Sensor *_sensor;
    Gate*   _gate;
    Controller *_controller;

    void process();
};
```

garagegate.h

```
int main()
{
    System sys;
    sys.process();

    return 0;
}
```

main.cpp

```
System::System()
{
    _gate   = new Gate(this, 5);
    _sensor = new Sensor(this);
    _engine = new Engine(this);
    _controller = new Controller(this);
}

System::~System()
{
    delete _gate   ;
    delete _sensor ;
    delete _engine ;
    delete _controller ;
}

void System::process()
{
    _controller->control();
}
```

garagegate.cpp

Vezérlő osztály

```
class Controller
{
public:
        Controller(System* s):_sys(s){}
        void control();

private:
        System* _sys;
        void menuWrite();
};
```

controller.h

```
enum Signal { none, off,
                up, down, stop,
                unrolled, coiled,
                activate, deactivate,
                blockage };
```

event.h

```
void Controller::control()
{
   menuWrite();
   int v = 0;
   do{
      std::cin >> v;  // ellenőrzés kell még
      switch(v){
         case 0: _sys->_engine->send(off);
                 _sys->_sensor->send(off);    break;
         case 1: _sys->_engine->send(up);     break;
         case 2: _sys->_engine->send(down); break;
         case 3: _sys->_engine->send(stop);   break;
      }

   }while(v != 0);
}
void Controller::menuWrite()
{
   std::cout << "Menupoints:\n";
   std::cout << "0 - exit\n";
   std::cout << "1 - up\n";
   std::cout << "2 - down\n";
   std::cout << "3 - stop\n";
}
```

szignálok küldése

controller.cpp

Kapu osztály

gate.h

```
class Gate
{
public:
	Gate(System* s, int m): _sys(s), _max_length(m), _cur_length(0) { }

	void up();
	void down();
	int getLength() const { return _cur_length; }
private:
	System* _sys;
	int _max_length;
	int _cur_length;
};
```

gate.cpp

```
void Gate::up()
{
	if(_cur_length>0) --_cur_length;
	if(0==_cur_length) _sys->_engine->send(coiled);
}
void Gate::down()
{
	if(_cur_length<_max_length) ++_cur_length;
	if(_max_length==_cur_length) _sys->_engine->send(unrolled);
}
```

ThreadSafeQueue osztálysablon

Mivel az Enqueue() és Dequeue() külön szálakon fut, szinkronizálni kell a működésüket, azaz a queue-t kölcsönösen kizárva használják.

```
class ThreadSafeQueue
{
private:
    std::queue<Signal> _queue;

    std::mutex _mu;
public:
    void enqueue(const Signal & e);
    {
        std::unique_lock<std::mutex> lock(_mu);
        _queue.push(e);
    }

    void dequeue(Signal & e)
    {
        std::unique_lock<std::mutex> lock(_mu);
        if(!empty() && _live){
            e = _queue.front();
            _queue.pop();
        }else e = Signal::none;
    }
    bool empty() const { return _queue.empty(); }
};
```

beépített queue<> sablonra épül
#include <queue>

szemafor kölcsönös kizárásos megvalósításához
#include <mutex>

kritikus szakasz eleje

kritikus szakasz vége

kritikus szakasz eleje

kritikus szakasz vége

Azonos mutex-ű lock és unlock (vagy blokk vége) közötti kritikus szakaszok közül egy időben legfeljebb egy szál futtathatja az egyik szakaszt.

threadsafequeue.h

Motor osztály

```
class Engine
{
public:
    Engine(System *s):_sys(s) {
        _eventQueue.startQueue();
        _thread = std::thread(&Engine::stateMachine, this);
    }
    ~Engine() {
        _currentState = finalstate;
        _eventQueue.stopQueue();
        _thread.join();
    }
   void send(Signal event) { _eventQueue.enqueue(event); }
private:
    System *_sys;
    ThreadSafeQueue<Signal> _eventQueue;
    enum State {finalstate, unmove, upward, downward
    State _currentState;
    std::thread _thread;

    void stateMachine();
    void transition();
    Signal getSignal();
};
```

külön szálon indul el
#include <thread>

megvárja, míg a külön indított szál befejeződik

külön szál az állapotgépnek

lásd állapot-átmenet tábla

```
void Engine::stateMachine()
{
    _currentState = unmove;
    while(_currentState!=finalstate) {
        transition();
        // várakozás
    }
}
```

engine.cpp

```
Signal Engine::getSignal()
{
    Signal signal;
    _eventQueue.dequeue(signal);
    return signal;
}
```

engine.cpp

engine.h